MISSION DE LA SÉANCE

Relier les données. Garder les preuves.

Une projection Neo4j reconstructible, puis un GraphRAG borné comparé honnêtement au RAG.



Data · graphe · GraphRAG

→ pour commencer

Photo Luke Chesser / Unsplash

LA CIBLE

Le graphe enrichit la réponse sans devenir une vérité magique.

SOURCE

Reconstructible

Les assertions et scripts restent versionnés hors Neo4j.

PARCOURS

Borné

Un à deux sauts et vingt relations maximum.

COMPARAISON

Honnête

GraphRAG n’est pas supposé gagner sur chaque question.

SE REPÉRER AVANT DE CODER

Un lien utile part d’un fait que l’on peut citer.



Relier des informations ne remplace pas leurs sources.

FOURNI

Le RAG S2 et le vocabulaire S4.

Réutiliser IDs métier et documents versionnés. Le jeu d’assertions musicothèque reste un secours.

À CRÉER

Une projection de relations sourcées.

Un fichier d’assertions décrit qui relie quoi, par quel prédicat et avec quelle preuve.

À COMPARER

Les mêmes questions avec et sans graphe.

Afficher les chemins utiles et les extraits ; mesurer aussi les cas où le graphe n’apporte rien.

Photo Luke Chesser / Unsplash · Pictogrammes : tracés Lucide, licence ISC.

PRÉPARER LES RELATIONS

Votre domaine S4 fournit les nœuds et le vocabulaire.

1

Lister les entités

Concepts dotés d’un identifiant métier stable.

2

Lister les relations

Verbes utiles pour répondre à des questions relationnelles.

3

Associer les preuves

Chaque relation factuelle possède une source et un extrait.

Installation complète

S5 réutilise le RAG, les contrats précédents et ajoute Neo4j.

```
uv sync --extra all --group dev
```

Sans données prêtes : utilisez temporairement `assertions.json`.

Neo4j est une projection, pas la source

DES FAITS EXPLICITES

Écrivez les relations avant de les charger.



- ✓ IDs lisibles et stables.
- ✓ Prédicats issus du vocabulaire métier.
- ✓ Source obligatoire pour tout fait.
- ✓ Petit fichier JSON versionnable.

ATELIER · ASSERTIONS VERS CYPHER

Commencez par une seule relation vérifiable.

01 **Écrire le fait et sa preuve**

Créer `data/sources/assertions.json`. Vérifier les IDs S4 et copier un extrait du corpus.

02 **Créer votre script de projection**

Créer `graph.cypher` ou un script Python paramétré. Contrainte sur Entity.id, puis MERGE.

03 **Tester l'idempotence**

Importer dans une base de TP vide, relever les comptes puis rejouer. Ne jamais vider une base partagée.

À CRÉER · assertions.json · exemple musicothèque

```
{
  "subjectId": "artist-amina-sol",
  "predicate": "PERFORMED_ON",
  "objectId": "track-blue-transit",
  "sourceId": "neon-harbor.md",
  "evidence": "Blue Transit accueille la trompettiste Amina
Sol."
}
# Le fichier contient une liste de ces faits.
```

Vérifier. Chaque relation retrouve un extrait dans une source. Les comptes restent identiques après le second import.

SCRIPT REJOUABLE

Construisez le graphe avec contraintes et MERGE.

01

Contrainte d’unicité

Un identifiant métier ne crée jamais deux nœuds.

02

MERGE

Le script peut être rejoué sans multiplier les relations.

03

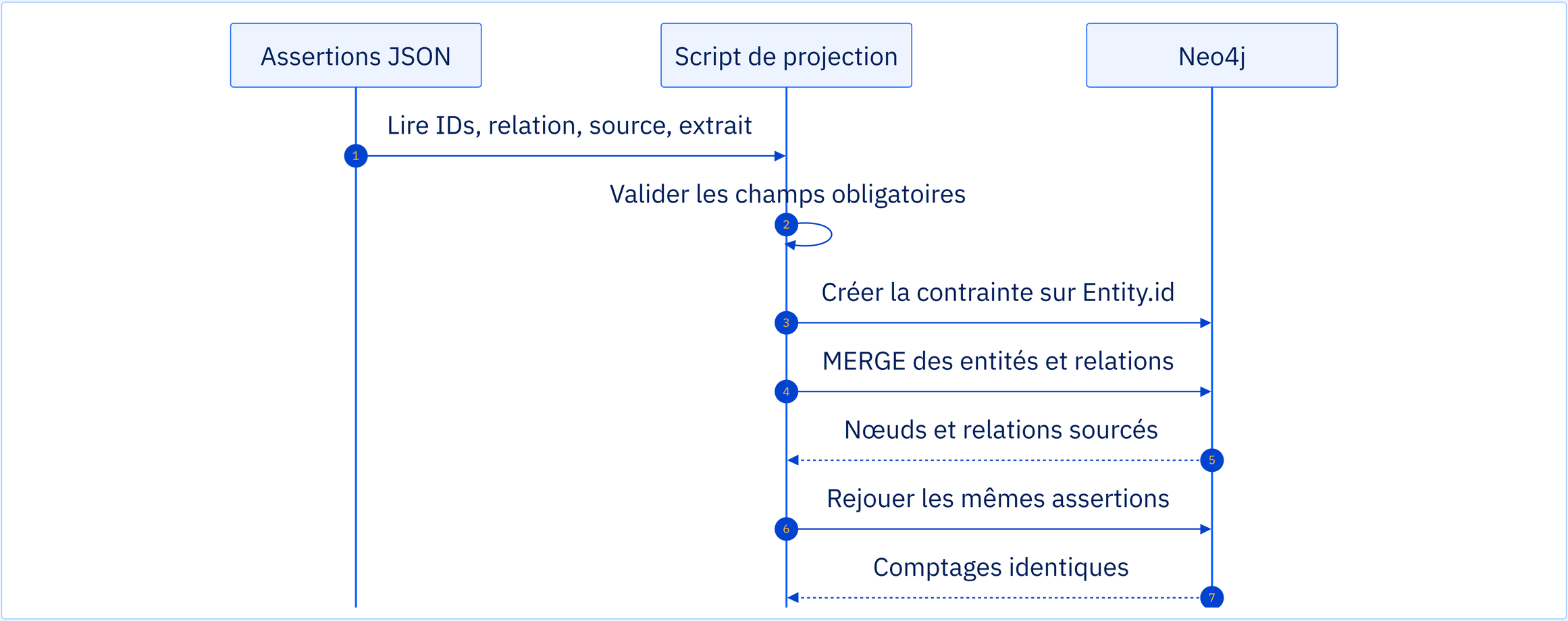
Provenance

`sourceId` et `evidence` voyagent avec la relation.

```
# Rejouer graph.cypher sur une base vide
# puis comparer le nombre de nœuds et de relations
```

FLUX À IMPLÉMENTER · SÉQUENCE

Le graphe doit pouvoir être reconstruit.



Les assertions restent la source versionnée. Neo4j reçoit des paramètres et des MERGE, avec provenance. Flèche pleine : appel ; pointillés : retour.

PAS DE CYPHER LIBRE

Le modèle choisit une intention, jamais une requête arbitraire.

Autorisé

- requêtes paramétrées connues ;
- un ou deux sauts ;
- vingt relations maximum ;
- résultats avec provenance.

Interdit

- Cypher généré puis exécuté ;
- IDs internes Neo4j ;
- traversée non bornée ;
- résultat sans source.

✓ Tester un chemin à un saut.

✓ Tester un chemin à deux sauts.

AVANT LA TRAVERSÉE

Reliez les mots de la question aux IDs du domaine.



A

Exact

Un nom ou alias reconnu retourne un ID connu.

B

Ambigu

Plusieurs candidats : demander ou conserver les options.

C

Absent

Aucun ID : retour explicite au RAG seul.

ATELIER · PARCOURS BORNÉ

Résolvez les IDs avant d’ouvrir la traversée.

01 **Définir un petit dictionnaire d’aliases**

Associer les noms connus aux IDs métier. Si aucun nom ne correspond, conserver le RAG seul.

02 **Écrire deux requêtes connues**

Une version un saut et une version deux sauts. Paramétrer l’ID et limiter les résultats à vingt.

03 **Tester trois issues**

ID connu avec chemin, alias inconnu, chemin absent. Vérifier aussi sourceId et extrait sur les relations retournées.

À CRÉER · adaptateur Neo4j · requête un saut

```
MATCH (a:Entity {id: $entity_id})
      -[r:RELATED_TO]-(b:Entity)
RETURN a.id, b.id, r.kind,
       r.sourceId, r.evidence
LIMIT $limit

# Paramètres contrôlés :
# entity_id connu ; 1 <= limit <= 20.
```

Vérifier. Le modèle ne produit jamais de Cypher exécuté librement. Adapter les labels à votre schéma, puis garder ce schéma cohérent partout.

DOCUMENTS + CHEMINS

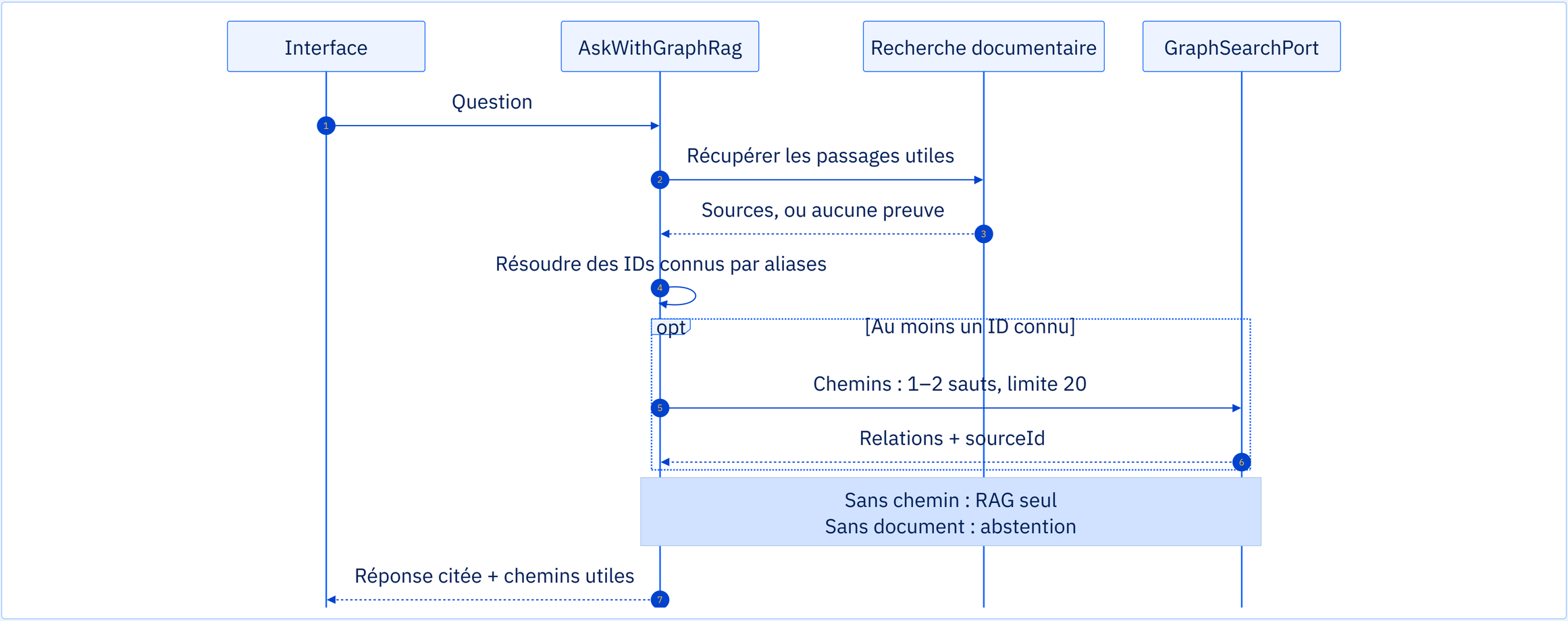
Le graphe relie. Les documents prouvent.

Recherche vectorielle + **Résolution d’entités** + **Chemins sourcés** → **Réponse citée**

- ✓ Aucun chemin utile → fallback RAG.
- ✓ Chemin sans preuve documentaire → abstention.
- ✓ Contexte total borné.
- ✓ Sources affichées à l’utilisateur.

FLUX À IMPLÉMENTER · SÉQUENCE

Les chemins enrichissent des preuves documentaires.



Les documents restent nécessaires. Sans ID ou chemin utile : RAG seul ; sans preuve documentaire : abstention. Flèche pleine : appel ; pointillés : retour.

Lire de haut en bas · les noms de ports désignent des contrats de l’application.

LA MÊME GRILLE POUR LES DEUX

RAG seul vs RAG + graphe : quatre questions, une conclusion.

QUESTION	RAG	GRAPHRAG
Question de sens	Réponse + sources	Gain attendu faible
Question de lien	Passages isolés	Chemin explicite
Question hybride	Preuves textuelles	Preuves + relation
Hors corpus	Abstention	Abstention

AVANT DE QUITTER S5

Le graphe reste borné, sourcé et reconstructible.

Vous avez construit

- des assertions sourcées ;
- un script Cypher rejouable ;
- des traversées bornées ;
- un GraphRAG avec fallback.

À livrer

- les quatre comparaisons ;
- les limites observées ;
- les scripts de reconstruction ;
- les tests et le README.

```
uv run pytest  &&  uv run ruff check .
git add .  &&  git commit -m "Ajoute le graphe sourcé et le GraphRAG borné"
```

S5 · CONSIGNES À GARDER PROJETÉES

À vous : des liens sourcés, un gain mesuré.

01 **Écrire les assertions**
Reprendre les IDs S4 ; chaque relation garde sourceId et extrait vérifiable.

02 **Reconstruire la projection**
Créer le script, contrainte et MERGE ; rejouer et comparer les comptes.

03 **Borner les parcours**
Aliases vers IDs connus ; requêtes paramétrées à 1–2 sauts, vingt relations au plus.

04 **Comparer les réponses**
Quatre questions RAG/GraphRAG ; afficher preuves, chemin utile, repli ou abstention.

À montrer avant de terminer

- Le même graphe après deux imports.
- Un chemin et son document justificatif.
- Le tableau RAG/GraphRAG, avec une limite observée.

```
uv run pytest
uv run ruff check .
```

Bloqué ? README de ce TP → « Pas à pas ». Secours : `tp/fixtures/music-library/assertions.json`. Sans graphe prêt, préserver le RAG fonctionnel.